

EXPERIMENT NO. 10

OSSP - Operating Systems and Systems Programming

Course Code: 25CS2104E

Task 1

Investigate Inode Structures Using stat(), ls -i, and find

```
/*
 * Experiment 10 - Task 1: Inode Information Investigation
 *
 * Demonstrates inspecting inode metadata using the stat() system call in C
 * and compares it with standard Unix commands (ls -i, stat, find).
 *
 * Inode Metadata fields displayed:
 * - Inode number (st_ino)
 * - File mode and permissions (st_mode)
 * - Hard link count (st_nlink)
 * - File size in bytes (st_size)
 * - Owner UID & GID (st_uid, st_gid)
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

void print_file_info(const char *filename) {
    struct stat file_stat;

    if (stat(filename, &file_stat) < 0) {
        perror("Error invoking stat()");
        exit(EXIT_FAILURE);
    }

    printf("=== Inode Investigation for '%s' ===\n", filename);
    printf("1. Inode Number:      %llu\n", (unsigned long long)file_stat.st_ino);
    printf("2. File Size:             %lld bytes\n", (long long)file_stat.st_size);
    printf("3. Hard Link Count:       %hu\n", (unsigned short)file_stat.st_nlink);
    printf("4. Raw Mode:              %o (octal)\n", file_stat.st_mode);
    printf("5. File Permissions:      ");
    printf((S_ISDIR(file_stat.st_mode)) ? "d" : "-");
    printf((file_stat.st_mode & S_IRUSR) ? "r" : "-");
    printf((file_stat.st_mode & S_IWUSR) ? "w" : "-");
    printf((file_stat.st_mode & S_IXUSR) ? "x" : "-");
    printf((file_stat.st_mode & S_IRGRP) ? "r" : "-");
    printf((file_stat.st_mode & S_IWGRP) ? "w" : "-");
    printf((file_stat.st_mode & S_IXGRP) ? "x" : "-");
    printf((file_stat.st_mode & S_IROTH) ? "r" : "-");
    printf((file_stat.st_mode & S_IWOTH) ? "w" : "-");
    printf((file_stat.st_mode & S_IXOTH) ? "x" : "-");
    printf("\n");
    printf("6. Owner User ID:         %u\n", file_stat.st_uid);
    printf("7. Owner Group ID:        %u\n", file_stat.st_gid);
    printf("8. Blocks Allocated:      %lld\n", (long long)file_stat.st_blocks);
    printf("=====\n");
}

int main(int argc, char *argv[]) {
    const char *target = "testfile.txt";
    if (argc >= 2) {
        target = argv[1];
    }
    print_file_info(target);
    return 0;
}
```

Output:

```

zsh - inode_investigation

rishivarma@MacBook-Air ~ % ./inode_investigation testfile.txt && ls -li testfile.txt
=== 1. C PROGRAM OUTPUT (stat system call) ===
=== Inode Investigation for 'testfile.txt' ===
1. Inode Number:      79020717
2. File Size:         177 bytes
3. Hard Link Count:   1
4. Raw Mode:          100644 (octal)
5. File Permissions: -rw-r--r--
6. Owner User ID:     501
7. Owner Group ID:    20
8. Blocks Allocated:  8
=====
=== 2. SHELL COMMAND: ls -li ===
79020717 testfile.txt
=== 3. SHELL COMMAND: stat ===
16777232 79020717 -rw-r--r-- 1 rishivarma staff 0 177 "Oct  4 19:16:23 2026" "Oct  4 19:16:17 2026" "Oct  4 19:16:22
=== 4. SHELL COMMAND: find . -inum 79020717 ===
./testfile.txt
rishivarma@MacBook-Air ~ %

```

Task 2

Hard Links and Inode Allocation

```

/*
 * Experiment 10 - Task 2: Hard Link Creation and Analysis
 *
 * Demonstrates:
 * - Checking initial inode and link count of original file (link count = 1).
 * - Creating a hard link using link() system call.
 * - Verifying both filenames share the same inode number.
 * - Verifying the hard link count increments from 1 to 2.
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>

void show_stat(const char *name) {
    struct stat st;
    if (stat(name, &st) < 0) {
        perror("stat failed");
        return;
    }
    printf(" %16s -> Inode: %llu | Links: %hu | Size: %lld bytes\n",
           name, (unsigned long long)st.st_ino, (unsigned short)st.st_nlink, (long long)
           st.st_size);
}

int main() {
    const char *orig = "original.txt";
    const char *hard = "hard_link.txt";

    printf("=== Hard Link Demonstration ===\n\n");

    // 1. Check original file before creating hard link
    printf("[Step 1] Status BEFORE creating hard link:\n");
    show_stat(orig);

    // 2. Remove existing hard link if present from previous run
    unlink(hard);

    // 3. Create hard link using link() system call
    printf("\n[Step 2] Creating hard link '%s' pointing to '%s' using link()...\n", hard
    , orig);
    if (link(orig, hard) < 0) {

```

```

        perror("Error creating hard link with link()");
        exit(EXIT_FAILURE);
    }
    printf("Hard link created successfully!\n\n");

    // 4. Verify both entries share the same inode and link count = 2
    printf("[Step 3] Status AFTER creating hard link:\n");
    show_stat(orig);
    show_stat(hard);

    printf("\nConclusion:\n");
    printf("- Both '%s' and '%s' refer to the exact same inode.\n", orig, hard);
    printf("- The link count increased from 1 to 2.\n");
    printf("- Changes made through either filename are immediately reflected in both.\n"
    );

    return 0;
}

```

Output:

```

zsh - hard_link_demo

rishivarma@MacBook-Air ~ % ./hard_link_demo && ls -li original.txt hard_link.txt
=== 1. C PROGRAM OUTPUT ===
=== Hard Link Demonstration ===
[Step 1] Status BEFORE creating hard link:
original.txt    -> Inode: 79020845 | Links: 1 | Size: 244 bytes
[Step 2] Creating hard link 'hard_link.txt' pointing to 'original.txt' using link()...
Hard link created successfully!
[Step 3] Status AFTER creating hard link:
original.txt    -> Inode: 79020845 | Links: 2 | Size: 244 bytes
hard_link.txt   -> Inode: 79020845 | Links: 2 | Size: 244 bytes
Conclusion:
- Both 'original.txt' and 'hard_link.txt' refer to the exact same inode.
- The link count increased from 1 to 2.
- Changes made through either filename are immediately reflected in both.
=== 2. SHELL COMMAND: ls -li ===
79020845 -rw-r--r--@ 2 rishivarma  staff  244 Oct  4 19:16 hard_link.txt
79020845 -rw-r--r--@ 2 rishivarma  staff  244 Oct  4 19:16 original.txt
=== 3. SHELL COMMAND: stat original.txt ===
16777232 79020845 -rw-r--r--  2 rishivarma  staff  0 244 "Oct  4 19:16:54 2026" "Oct  4 19:16:48 2026" "Oct  4 19:17:03
=== 4. SHELL COMMAND: stat hard_link.txt ===
16777232 79020845 -rw-r--r--  2 rishivarma  staff  0 244 "Oct  4 19:16:54 2026" "Oct  4 19:16:48 2026" "Oct  4 19:17:03
rishivarma@MacBook-Air ~ %

```

Task 3

Symbolic Links and Inode Allocation

```

/*
 * Experiment 10 - Task 3: Symbolic (Soft) Link Creation and Analysis
 *
 * Demonstrates:
 * - Creating a symbolic link using the symlink() system call.
 * - Inspecting inode difference using lstat() vs stat().
 * - Reading the symlink destination using readlink().
 * - Observing broken / dangling link behavior when target is deleted (unlink).
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <errno.h>
#include <string.h>

void print_lstat(const char *name) {
    struct stat st;

```

```
    if (lstat(name, &st) < 0) {
        perror("lstat failed");
        return;
    }
    printf("  %-18s -> Inode: %llu | Links: %hu | Type: %s\n",
        name, (unsigned long long)st.st_ino, (unsigned short)st.st_nlink,
        S_ISLNK(st.st_mode) ? "Symbolic Link" : "Regular File");
}

int main() {
    const char *orig = "original_target.txt";
    const char *sym = "soft_link.txt";

    printf("=== Symbolic (Soft) Link Demonstration ===\n\n");

    // 1. Remove any old symlink
    unlink(sym);

    // 2. Create symbolic link using symlink() system call
    printf("[Step 1] Creating symbolic link '%s' pointing to '%s'...\n", sym, orig);
    if (symlink(orig, sym) < 0) {
        perror("Error creating symbolic link with symlink()");
        exit(EXIT_FAILURE);
    }
    printf("Symbolic link created successfully!\n\n");

    // 3. Inspect inodes using lstat()
    printf("[Step 2] Comparing inodes of original file and symbolic link:\n");
    print_lstat(orig);
    print_lstat(sym);

    // 4. Read link target string using readlink() system call
    char target_path[256];
    ssize_t len = readlink(sym, target_path, sizeof(target_path) - 1);
    if (len > 0) {
        target_path[len] = '\0';
        printf("\n[Step 3] readlink() result: '%s' points to '%s'\n\n", sym, target_path);
    }

    // 5. Test opening via symbolic link while target exists
    printf("[Step 4] Reading data through symbolic link '%s':\n", sym);
    FILE *fp = fopen(sym, "r");
    if (fp) {
        char line[128];
        if (fgets(line, sizeof(line), fp)) {
            printf("  First line: %s", line);
        }
        fclose(fp);
    } else {
        perror("Failed to open symlink");
    }

    // 6. Delete target file to demonstrate dangling/broken symlink
    printf("\n[Step 5] Deleting original target '%s' (simulating broken link)...\n", orig);
    unlink(orig);

    // 7. Try opening symlink now
    printf("[Step 6] Attempting to open '%s' after target was deleted:\n", sym);
    FILE *fp_broken = fopen(sym, "r");
    if (fp_broken == NULL) {
        printf("  Result: FAILED as expected! Error: %s (errno: %d)\n", strerror(errno),
            errno);
        printf("  The symbolic link is now broken / dangling!\n");
    } else {
        fclose(fp_broken);
    }

    // 8. Recreate target file so workspace remains clean and verifiable
    printf("\n[Step 7] Restoring original target file '%s'...\n", orig);
    FILE *fp_restored = fopen(orig, "w");
    if (fp_restored) {
        fputs("Operating Systems and Systems Programming (OSSP) Lab.\n", fp_restored);
        fputs("Experiment 10 Task 3: Symbolic Link (Soft Link) Demonstration.\n", fp_restored);
    }
}
```

```

        fclose(fp_restored);
        printf("Target restored! Symbolic link is valid again.\n");
    }

    return 0;
}

```

Output:

```

zsh - symlink_demo

rishivarma@MacBook-Air ~ % ./symlink_demo && ls -li original_target.txt soft_link.txt
=== 1. C PROGRAM OUTPUT ===
=== Symbolic (Soft) Link Demonstration ===
[Step 1] Creating symbolic link 'soft_link.txt' pointing to 'original_target.txt'...
Symbolic link created successfully!
[Step 2] Comparing inodes of original file and symbolic link:
original_target.txt -> Inode: 79020986 | Links: 1 | Type: Regular File
soft_link.txt       -> Inode: 79021104 | Links: 1 | Type: Symbolic Link
[Step 3] readlink() result: 'soft_link.txt' points to 'original_target.txt'
[Step 4] Reading data through symbolic link 'soft_link.txt':
First line: Operating Systems and Systems Programming (OSSP) Lab.
[Step 5] Deleting original target 'original_target.txt' (simulating broken link)...
[Step 6] Attempting to open 'soft_link.txt' after target was deleted:
Result: FAILED as expected! Error: No such file or directory (errno: 2)
The symbolic link is now broken / dangling!
[Step 7] Restoring original target file 'original_target.txt'...
Target restored! Symbolic link is valid again.
=== 2. SHELL COMMAND: ls -li ===
79021105 -rw-r--r--@ 1 rishivarma  staff  117 Oct  4 19:17 original_target.txt
79021104 lrwxr-xr-x@ 1 rishivarma  staff   19 Oct  4 19:17 soft_link.txt -> original_target.txt
=== 3. SHELL COMMAND: readlink soft_link.txt ===
original_target.txt
=== 4. SHELL COMMAND: stat soft_link.txt ===
16777232 79021104 lrwxr-xr-x 1 rishivarma staff 0 19 "Oct  4 19:17:34 2026" "Oct  4 19:17:34 2026" "Oct  4 19:17:34
rishivarma@MacBook-Air ~ %

```

Task 4

Memory-Mapped File Reading (mmap)

```

/*
 * Experiment 10 - Task 4: File Reading using mmap()
 *
 * System Calls Demonstrated:
 * - open(): Opens the file in read-only mode.
 * - fstat(): Retrieves file status and total file size.
 * - mmap(): Maps the file directly into the process's virtual address space.
 * - munmap(): Unmaps the memory region.
 * - close(): Closes the file descriptor.
 */

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    const char *filename = "sample_data.txt";
    if (argc >= 2) {
        filename = argv[1];
    }

    printf("=== Memory Mapped File Reading (mmap) ===\n");
    printf("Target File: %s\n", filename);

    // 1. open() file
    int fd = open(filename, O_RDONLY);

```

```
    if (fd < 0) {
        perror("Error opening file");
        exit(EXIT_FAILURE);
    }
    printf("[1] open(): File opened successfully (fd = %d)\n", fd);

    // 2. fstat() to obtain file size
    struct stat sb;
    if (fstat(fd, &sb) < 0) {
        perror("Error getting file stats with fstat");
        close(fd);
        exit(EXIT_FAILURE);
    }
    printf("[2] fstat(): File size is %lld bytes\n", (long long)sb.st_size);

    if (sb.st_size == 0) {
        printf("File is empty, cannot mmap 0 bytes.\n");
        close(fd);
        return 0;
    }

    // 3. mmap() into process address space
    char *mapped_data = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (mapped_data == MAP_FAILED) {
        perror("Error mapping memory with mmap");
        close(fd);
        exit(EXIT_FAILURE);
    }
    printf("[3] mmap(): File successfully mapped at virtual memory address: %p\n", (void
        *)mapped_data);

    // 4. Read directly from memory pointer (zero-copy read)
    printf("\n[4] Displaying mapped file content directly from memory:\n");
    printf("-----\n");
    fflush(stdout);
    write(STDOUT_FILENO, mapped_data, sb.st_size);
    printf("-----\n\n");
    fflush(stdout);

    // 5. munmap() to release mapped memory
    if (munmap(mapped_data, sb.st_size) < 0) {
        perror("Error unmapping memory with munmap");
    } else {
        printf("[5] munmap(): Memory successfully unmapped.\n");
    }

    // 6. close() file descriptor
    close(fd);
    printf("[6] close(): File descriptor closed.\n");

    return 0;
}
```

Output:

```

zsh - mmap_read

rishivarma@MacBook-Air ~ % ./mmap_read sample_data.txt
=== Memory Mapped File Reading (mmap) ===
Target File: sample_data.txt
[1] open(): File opened successfully (fd = 3)
[2] fstat(): File size is 307 bytes
[3] mmap(): File successfully mapped at virtual memory address: 0x102388000
[4] Displaying mapped file content directly from memory:

=====
Operating Systems and Systems Programming (OSSP)
Experiment 10 Task 4: Memory Mapped File Reading
Memory mapping maps disk blocks directly into the virtual address space.
Zero-copy read eliminates user/kernel buffer copying!
=====

[5] munmap(): Memory successfully unmapped.
[6] close(): File descriptor closed.
rishivarma@MacBook-Air ~ %

```

Task 5**Memory-Mapped File Writing (mmap & msync)**

```

/*
 * Experiment 10 - Task 5: File Writing using mmap()
 *
 * System Calls Demonstrated:
 * - open(): Opens/creates a file with read/write mode (O_RDWR | O_CREAT | O_TRUNC).
 * - ftruncate(): Expands/sets file size on disk before mapping.
 * - mmap(): Maps file pages into memory with PROT_READ | PROT_WRITE and MAP_SHARED.
 * - sprintf()/memcpy(): Writes data directly into memory address space.
 * - msync(): Flushes/synchronizes dirty mapped pages to permanent disk storage.
 * - munmap(): Releases memory mapping.
 * - close(): Closes file descriptor.
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

#define FILE_SIZE 256

int main(int argc, char *argv[]) {
    const char *filename = "mapped_output.txt";
    if (argc >= 2) {
        filename = argv[1];
    }

    printf("=== Memory Mapped File Writing (mmap & msync) ===\n");
    printf("Target File: %s\n\n", filename);

    // 1. open() with read/write permissions
    int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("Error opening/creating file");
        exit(EXIT_FAILURE);
    }
    printf("[1] open(): File created and opened in O_RDWR mode (fd = %d)\n", fd);

    // 2. ftruncate() to allocate size for the file
    if (ftruncate(fd, FILE_SIZE) < 0) {
        perror("Error truncating file");
        close(fd);
    }

```

```

        exit(EXIT_FAILURE);
    }
    printf("[2] ftruncate(): File size set to %d bytes\n", FILE_SIZE);

    // 3. mmap() with read/write and MAP_SHARED
    char *map_ptr = mmap(NULL, FILE_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (map_ptr == MAP_FAILED) {
        perror("Error mapping memory with mmap");
        close(fd);
        exit(EXIT_FAILURE);
    }
    printf("[3] mmap(): Memory mapped successfully at %p with MAP_SHARED\n", (void *)map_ptr);

    // 4. Write data directly into memory
    const char *message =
        "Operating Systems and Systems Programming (OSSP) Lab.\n"
        "Experiment 10 Task 5: Memory Mapped File Writing.\n"
        "Data written directly into memory pointer using mmap()!\n"
        "Synchronized to disk using msync(MS_SYNC).\n";

    memset(map_ptr, 0, FILE_SIZE);
    strncpy(map_ptr, message, FILE_SIZE - 1);
    printf("[4] Memory Write: Text written into memory pointer.\n");

    // 5. msync() to flush dirty pages to disk
    if (msync(map_ptr, FILE_SIZE, MS_SYNC) < 0) {
        perror("Error synchronizing memory with msync");
    } else {
        printf("[5] msync(): Memory modifications successfully flushed to disk storage.\n");
    }

    // 6. munmap() to release mapping
    if (munmap(map_ptr, FILE_SIZE) < 0) {
        perror("Error unmapping memory");
    } else {
        printf("[6] munmap(): Memory successfully unmapped.\n");
    }

    // 7. close() file descriptor
    close(fd);
    printf("[7] close(): File descriptor closed.\n\n");

    printf("File write operation complete! You can inspect '%s'.\n", filename);
    return 0;
}

```

Output:

```

zsh - mmap_write

rishivarma@MacBook-Air ~ % ./mmap_write mapped_output.txt && cat mapped_output.txt
=== Memory Mapped File Writing (mmap & msync) ===
Target File: mapped_output.txt
[1] open(): File created and opened in O_RDWR mode (fd = 3)
[2] ftruncate(): File size set to 256 bytes
[3] mmap(): Memory mapped successfully at 0x100798000 with MAP_SHARED
[4] Memory Write: Text written into memory pointer.
[5] msync(): Memory modifications successfully flushed to disk storage.
[6] munmap(): Memory successfully unmapped.
[7] close(): File descriptor closed.
File write operation complete! You can inspect 'mapped_output.txt'.
=== CONTENTS OF mapped_output.txt ===
Operating Systems and Systems Programming (OSSP) Lab.
Experiment 10 Task 5: Memory Mapped File Writing.
Data written directly into memory pointer using mmap()!
Synchronized to disk using msync(MS_SYNC).
rishivarma@MacBook-Air ~ %

```